Universidad de Alcalá | INGENIERIA DEL SOFTWARE

PL1 INGENIERÍA DEL SOFTWARE

Pablo Hernández Borregón | 09823707Q

Mario Garrido Pradillo | 02760341L

INDICE:

1. **Introducción**
2. **Especificación de Requisitos Software**
3. **Documentos de Casos de Uso**
4. **Diagrama de Casos de Uso**
5. **Diagrama de Interfaz**
6. **Diagramas de Secuencias**
7. **Diagrama de Clases de Diseño.**
8. **Diagramas de Componentes y de Despliegue.**
9. **Trazabilidad de un Requisito Funcional en el proyecto.**
10. **Conclusiones**

1 INTRODUCCION

El presente documento detalla la especificación y diseño de la aplicación informática "JavaEat", encargada por una empresa que busca facilitar la venta de comida de diversos restaurantes. Esta aplicación se desarrollará en Java, utilizando el entorno Java SE y la interfaz gráfica Swing. La finalidad principal es crear una herramienta intuitiva y eficiente que satisfaga las necesidades de dos tipos de usuarios: el Administrador de la aplicación y los Clientes (particulares y empresas).

La aplicación gestionará usuarios, restaurantes y pedidos, permitiendo a los clientes buscar, seleccionar y comprar comidas de los restaurantes registrados. Además, se incluirá la funcionalidad de opinar sobre los restaurantes y la posibilidad de que las empresas realicen pedidos con un 10% de descuento y soliciten servicios de catering.

A continuación, se detallarán los principales requisitos y funcionalidades, así como los aspectos técnicos y de diseño que se deben tener en cuenta para el desarrollo de "JavaEat". Este documento también contendrá información sobre la estructura del código, la separación de clases, la gestión de excepciones y otros aspectos importantes para garantizar un desarrollo robusto y fácilmente mantenible.

Además, se incluirán los diagramas de casos de uso, diagrama de interfaz y diagramas de secuencias, elaborados mediante técnicas de ingeniería inversa UML con la herramienta Enterprise Architect, para proporcionar una visión visual y clara de la arquitectura y el flujo de la aplicación.

2 ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

Introducción

La aplicación "JavaEat" tiene como objetivo proporcionar una plataforma para la venta de comida de diversos restaurantes. Este documento de Especificación de Requisitos de Software (SRS) detalla los requisitos funcionales y no funcionales para el desarrollo de la aplicación. Está dirigido a desarrolladores, diseñadores y cualquier persona involucrada en el proceso de creación de la aplicación.

Propósito

- Este documento tiene como propósito definir los requisitos funcionales y no funcionales de la aplicación "JavaEat".
- Está destinado a desarrolladores, diseñadores, probadores y cualquier otra persona involucrada en el ciclo de vida del desarrollo de software.

Alcance

- El producto a desarrollar es la aplicación informática "JavaEat", diseñada para la gestión de pedidos y venta de comida de diversos restaurantes.
- Este documento es consistente con otros documentos de mayor nivel, como la Descripción del Sistema, que pueda haber en el contexto del desarrollo de software.

Personal Involucrado

Nombre	Administrador del proyecto
Rol	Administrar, gestionar y supervisar el proyecto
Categoría profesional	Gestión de proyectos
Responsabilidades	Coordinación general del proyecto
Información de contacto	Correo del administrador, Número de teléfono
Aprobación	Debe aprobar el proyecto

Nombre	Desarrolladores
Rol	Desarrollar el software de forma organizada
Categoría profesional	Desarrollo de aplicaciones
Responsabilidades	Implementar las funcionalidades, codificación y solución de problemas técnicos
Información de contacto	Correos electrónicos, Números de teléfono
Aprobación	El administrador del proyecto debe aprobar el trabajo de los desarrolladores

Nombre	Diseñadores de la interfaz
Rol	Diseñar las interfaces de la aplicación de JavaEat
Categoría profesional	Diseño Gráfico, Mercadotecnia y Experiencia de usuario
Responsabilidades	Crear interfaces visuales intuitivas y atractivas, y diseñar los elementos gráficos
Información de contacto	Correos electrónicos, Números de teléfono
Aprobación	El administrador del proyecto debe aprobar el trabajo de los diseñadores de la interfaz

Definiciones, Acrónimos y Abreviaturas

- Definiciones
  JavaEat: Nombre de la aplicación.
  SRS: Especificación de Requisitos de Software.
- Acrónimos
  CP: Código Postal.
  DNI: Documento Nacional de Identidad.
  CIF: Código de Identificación Fiscal.
- Abreviaturas
  SE: Java SE (Standard Edition).

Descripción General

Perspectiva del Producto

La aplicación "JavaEat" es un producto independiente diseñado para facilitar la venta de comida de diversos restaurantes. Se sitúa como una solución integral para la gestión de usuarios, restaurantes, y pedidos en el ámbito de venta de alimentos.

Funcionalidad del Producto

La aplicación JavaEat debe incluir las siguientes funcionalidades principales:

- Registro e inicio de sesión para administradores y clientes.

- Gestión de usuarios: administración y mantenimiento de datos de clientes particulares y empresas.
- Gestión de restaurantes: registro y mantenimiento de información detallada de cada restaurante.
- Realización y gestión de pedidos: permitir a los clientes realizar pedidos a los restaurantes registrados.
- Valoración y calificación de restaurantes por parte de los clientes.
- Búsqueda y filtrado de restaurantes por especialidad.
- Realización de pedidos con herramienta de búsqueda por código postal y opciones de ordenación.
- Interfaz gráfica intuitiva y fácil de usar con separación clara entre clases del dominio y la interfaz de usuario.
- Generación automática de factura de venta y almacenamiento de datos de venta.

Características de los Usuarios

Tipo de usuario	Administrador
Formación	Personal autorizado de la empresa
Habilidades	Gestión de empresas, y ventas
Actividades	Control de ventas, restaurantes y usuarios registrados

Tipo de usuario	Cliente particular
Formación	Cualquiera
Habilidades	Consumidor
Actividades	Registro, búsqueda de restaurantes, pedidos de comida, y gestión de datos personales

Tipo de usuario	Empresa
Formación	Cualquiera
Habilidades	Consumidor
Actividades	Registro, búsqueda de restaurantes, pedidos de comida con o sin catering, y gestión de datos personales

Restricciones

Utilización del lenguaje Java compatible con Java SE.
Tratamiento correcto de excepciones.
Separación estricta entre clases del dominio y la interfaz de usuario.
Almacenamiento en archivos para datos, prohibido el uso de bases de datos.

Evolución Previsible del Sistema

Se prevé que el sistema pueda evolucionar mediante futuras mejoras como la expansión a plataformas web o móviles, incorporación de nuevas funcionalidades y optimización del rendimiento.

Requisitos Específicos

Número de requisito	RF_001
Nombre de requisito	Registro de cliente
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	El cliente puede ser un particular y una empresa, y en el caso de que sea la primere vez que usa la aplicación deberá registrarse introduciendo los datos requeridos para cada tipo de cliente.

Número de requisito	RF_002
Nombre de requisito	Inicio de sesión
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☒Seguridad ☐Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	A la hora de entrar a la aplicación se deben introducir las credenciales propias del actor que sea, el administrador tiene unas credenciales fijas comunes para todos los administradores, pero los clientes deben introducir las credenciales que introdujeron en el RF_001.

Número de requisito	RF_003
Nombre de requisito	Dar de alta a los restaurantes
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Los administradores deben de poder dar de alta a los restaurantes que vayan a aparecer posteriormente en la aplicación de cara a la vista de los clientes, introduciendo los datos necesarios para cada restaurante.

Número de requisito	RF_004
Nombre de requisito	Añadir comidas a restaurantes
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Los administradores dentro de cada restaurante deben poder añadir comidas para que se puedan ejecutar las ventas, estas deben de registrar las descripciones, los ingredientes y el precio de cada una.

Número de requisito	RF_005
Nombre de requisito	Servicio de catering
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Los clientes de tipo empresa deben poder seleccionar o no un servicio de cátering cuando vayan a pedir alguna comida a algún restaurante que lo ofrezca. Este servicio lo tiene que activar el administrador, pero es la empresa la que decide si ofrecer el servicio o no.

Número de requisito	RF_006
Nombre de requisito	Gestionar usuarios
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Los administradores deben poder consultar los usuarios que se han

	registrado en la aplicación y los clientes deben poder gestionar sus datos personales

Número de requisito	RF_007
Nombre de requisito	Búsqueda de restaurantes
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Los clientes deben poder buscar los restaurantes filtrando por código postal y especialidad.

Número de requisito	RF_008
Nombre de requisito	Filtrar restaurantes
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Se deben poder filtrar restaurantes por calificación, tiempo de envío o disponibilidad de cátering en el caso de ser una empresa.

Número de requisito	RF_009
Nombre de requisito	Seleccionar comidas
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	Los clientes deben poder seleccionar comidas añadiéndolas al carrito.

Número de requisito	RNF_010
Nombre de requisito	Calcular precios
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	Una vez se añadan comidas al carrito se calculará el precio total del pedido antes de realizarlo.

Número de requisito	RF_011
Nombre de requisito	Realizar pedido
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	Los clientes deben poder efectuar el pago para cerrar el pedido que quieran hacer.

Número de requisito	RNF_012
Nombre de requisito	Descuento para empresas
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☐ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	Las empresas obtendrán un 10% de descuento

Número de requisito	RNF_013
Nombre de requisito	Generar facturas
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Al realizar el pedido se imprimirá una factura del mismo

Número de requisito	RNF_014
Nombre de requisito	Almacenar pedidos
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Se guardarán todos los datos de cada venta

Número de requisito	RNF_015
Nombre de requisito	Interfaz gráfica
Tipo	☐Funcional ☐Datos ☐Rendimiento ☒Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	La aplicación se realizará con una interfaz desarrollada con swing

Número de requisito	RNF_016
Nombre de requisito	Logo de Javaeat
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☐ Alta/Esencial ☐ Media/Deseado ☒ Baja/ Opcional
Descripción	Cada ventana debe incluir el anagrama de javaeat

Número de requisito	RNF_017
Nombre de requisito	Seguridad de acceso
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☒Seguridad ☐Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	Si las credenciales introducidas en el inicio de sesión no coinciden con las de algún usuario ya registrado no dejara acceder a la aplicación

Número de requisito	RNF_018
Nombre de requisito	Lenguaje Java
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	La aplicación debe estar desarrollada en el lenguaje Java y debe ser compatible con Java SE

Número de requisito	RNF_019
Nombre de requisito	Control de clases
Tipo	☐Funcional ☐Datos ☒Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	No se deben usar clases o métodos obsoletos

Número de requisito	RF_020
Nombre de requisito	Guardar datos
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	Al acabar de usarse la aplicación se deben guardar todos los datos y cambios efectuados, asimismo al volver a entrar en la aplicación todos los datos del ultimo uso estarán disponibles

Número de requisito	RNF_021
Nombre de requisito	Excepciones
Tipo	☐Funcional ☐Datos ☒Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	Se deberá realizar un correcto tratamiento de excepciones.

Número de requisito	RNF_022
Nombre de requisito	Interfaz intuitiva
Tipo	☐Funcional ☐Datos ☐Rendimiento ☒Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	La interfaz será intuitiva y fácil de usar

Número de requisito	RNF_023
Nombre de requisito	Separación de clases
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Se separaran las clases de dominio, las de gestión de interfaz por paquetes y las de modelo.

Número de requisito	RNF_024
Nombre de requisito	Serialización
Tipo	☐Funcional ☒Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional
Descripción	Se deberán almacenar los datos en archivos, no se permiten el uso de bases de datos

Número de requisito	RNF_025
Nombre de requisito	Fácil mantenibilidad
Tipo	☐Funcional ☐Datos ☒Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional
Descripción	Cualquier programador deberá poder mantener el sistema fácilmente en Java, Empleando comentarios y una buena organización para facilitar el entendimiento.

Número de requisito	RNF_026
Nombre de requisito	Tamaño mínimo para las pruebas
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☐ Alta/Esencial ☐ Media/Deseado ☒ Baja/ Opcional
Descripción	Para poder comprobar el correcto funcionamiento de la aplicación se deberá poder arrancar con un mínimo de 3 usuarios, 4 restaurantes y 12 comidas (3 por restaurante)

Número de requisito	RNF_027
Nombre de requisito	UML
Tipo	☐Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☒Otros
Prioridad del requisito	☐ Alta/Esencial ☐ Media/Deseado ☒ Baja/ Opcional

Descripción	Se debe realizar un diagrama UML mediante ingeniería inversa.

Número de requisito	RF_028
Nombre de requisito	Opinion de restaurantes
Tipo	☒Funcional ☐Datos ☐Rendimiento ☐Interfaz ☐Seguridad ☐Otros
Prioridad del requisito	☐ Alta/Esencial ☐ Media/Deseado ☒ Baja/ Opcional
Descripción	Se debe de poder opinar y calificar los restaurantes

Requisitos Comunes de los Interfaces

RF_001: Registro de Cliente
Entrada: Datos requeridos para cada cliente
Salida: Confirmación de registro
RF_002: Inicio de Sesión
Entrada: Credenciales del usuario.
Salida: Acceso permitido o denegado
RF_003: Dar de alta restaurantes
Entrada: Datos necesarios de cada restaurante
Salida: confirmación del alta del restaurante
RF_004: Añadir comidas a los restaurantes
Entrada: Descripciones de estas, ingredientes y precios
Salida: Confirmación de la comida
RF_005: Servicio de Catering
Entrada: Decisión sobre el servicio de catering
Salida: Activación del servicio según la decisión tomada por la empresa
RF_006: Gestionar usuarios
Entrada: Consulta de los usuarios registrados
Salida: Lista de estos usuarios
RF_007: Búsqueda de restaurantes
Entrada: Código postal y especialidad de restaurantes
Salida: Lista de restaurantes concorde a los criterios
RF_008: Filtrar restaurantes
Entrada: Calificación, disponibilidad de catering, tiempo de envío
Salida: Lista de restaurantes filtrados
RF_009: Seleccionar comidas
Entrada: Selección de comidas para añadir al carrito
Salida: Confirmación de la selección
RF_011: Realizar pedido
Entrada: Proceso de pago
Salida: Confirmación de pedido realizado
RF_020: Guardar datos
Entrada: Datos y cambios efectuados dentro de la aplicación
Salida: Datos guardados y disponibles para próximo uso
RF_028: Opinion de restaurantes
Entrada: El usuario pone su opinión y su valoración
Salida: El sistema añade la opinión y hace una media de las calificaciones añadiendo la nueva
RNF_10: Calcular precios
Entrada: El usuario añade comidas al carrito.
Salida: El sistema devuelve el precio total de las comidas
RNF_012: descuento para empresas
Entrada: Identificación de empresas
Salida: Aplicación del descuento
RNF_013: Generar facturas
Entrada: Detalles del pedido
Salida: Factura generada
RNF_014: Almacenar pedidos

Entrada: Datos de cada venta
Salida: Almacenamiento de datos de venta
RNF_016: Logo JavaEat
Entrada: Presencia del anagrama
Salida: Visualización de este en cada ventana
RNF_017: Seguridad de acceso
Entrada: Credenciales de inicio de sesión
Salida: Acceso permitido o denegado
RNF_022: Interfaz intuitiva
Entrada: Diseño de la interfaz
Salida: Interfaz fácil de usar
RNF_024: Serialización
Entrada: Datos a almacenar
Salida: Almacenamiento de datos en archivos
RNF_025: Fácil mantenibilidad
Entrada: Organización y comentarios en el código
Salida: Sistema fácilmente mantenible
RNF_026: Tamaño mínimo para las pruebas
Entrada: Número mínimo de usuarios, restaurantes y comidas
Salida: Ejecución de pruebas con los datos mínimos
RNF_027: UML
Entrada: Diagrama UML generado mediante ingeniería inversa
Salida: Diagrama UML del sistema

Interfaces de Usuario

Interfaz Gráfica (RNF_015)

- Este requisito establece que la aplicación debe contar con una interfaz gráfica que será desarrollada utilizando la tecnología Swing

Interfaz Intuitiva (RNF_022)

- Este requisito especifica que la interfaz de la aplicación debe ser intuitiva y fácil de usar, lo que contribuirá a una mejor experiencia del usuario. Además, se espera que esta interfaz tenga un buen rendimiento en términos de usabilidad.

Interfaces de Hardware

Excepciones (RF_021)

- Este requisito menciona el tratamiento de excepciones, que podría estar relacionado con el manejo de errores a nivel de sistema, incluidos problemas de hardware.

Serialización (RNF_024)

- Aunque este requisito se centra en el almacenamiento de datos en archivos en lugar de bases de datos, la serialización también puede estar relacionada con la interacción del software con dispositivos de almacenamiento o comunicación de hardware.

Interfaces de Software

Logo JavaEat (RF_016)

- Se requiere algún software de diseño gráfico para crear y manipular el logo de JavaEat.

- El propósito del interfaz es proporcionar una identificación visual de la marca JavaEat en la interfaz de usuario de la aplicación.

- El interfaz consiste en la inclusión del logo de JavaEat en cada ventana de la aplicación.

Interfaces de Comunicación

Inicio de sesión (RF_002)

- A la hora de entrar a la aplicación se deben introducir las credenciales propias del actor que sea. Los administradores tienen credenciales fijas comunes para todos, mientras que los clientes deben usar las credenciales introducidas en el requisito de "Registro de cliente”.

- La comunicación aquí implica la verificación de las credenciales ingresadas con respecto a la base de datos de usuarios registrados en la aplicación.

- Se utilizan protocolos seguros como HTTPS para la transmisión segura de credenciales.

Realizar Pedido (RF_011)

- Los clientes deben poder efectuar el pago para cerrar el pedido que quieran hacer.

- Este requisito implica la comunicación con sistemas de procesamiento de pagos para realizar transacciones financieras.

- El protocolo de pago común como HTTPS pueden utilizarse para la comunicación segura con los servicios de procesamiento de pagos.

Generar Pedido (RNF_013)

- Al realizar el pedido se imprimirá una factura de este.

- Este requisito implica la comunicación con sistemas de generación de facturas o impresión.

- No se especifica un protocolo de comunicación específico, pero podría implicar la generación de documentos en un formato estándar como PDF o HTML para su posterior impresión.

Opinion de restaurantes(RF_028)

- El usuario podrá añadir una opinión y una valoración de cada restaurante lo que podrá afectar en la calificación del mismo

Requisitos Funcionales

Requisito Funcional 1

Registro de cliente (RF_001)

- Recepción de información: El software debe recibir datos de registro de clientes, incluyendo información específica para usuarios particulares y empresas.
- Comprobación de validez de las entradas: Verificación de que los datos ingresados cumplen con los requisitos de formato y validez.
- Generación de salidas: Creación de registros de clientes en la base de datos.
- Especificación de requisitos lógicos para la información almacenada en la base de datos: Debe incluir campos como nombre, dirección, tipo de cliente (particular o empresa), entre otros.

Requisito Funcional 2

Inicio de sesión (RF_002)

- Recepción de información: El software debe recibir credenciales de inicio de sesión.
- Comprobación de validez de las entradas: Verificación de que las credenciales ingresadas sean válidas y correspondan a un usuario registrado.
- Generación de salidas: Permitir el acceso al sistema si las credenciales son válidas.
- Respuesta a situaciones anormales: Manejo de errores en caso de credenciales incorrectas.
- Parámetros: Identificación del tipo de usuario (administrador o cliente) para dirigir a la interfaz correspondiente.

Requisito Funcional 3

Dar de alta a los restaurantes (RF_003)

- Recepción de información: El software debe recibir datos para dar de alta un restaurante, como nombre, dirección, especialidad, etc.
- Comprobación de validez de las entradas: Verificación de que los datos ingresados sean válidos y completos.
- Generación de salidas: Creación de registros de restaurantes en la base de datos.

Requisito Funcional 4

Añadir comidas a restaurantes (RF_004)

- Recepción de información: El software debe recibir datos para añadir comidas a un restaurante, incluyendo descripciones, ingredientes y precios.
- Comprobación de validez de las entradas: Verificación de que los datos ingresados sean válidos y completos.
- Generación de salidas: Actualización de la información sobre comidas en la base de datos del restaurante correspondiente.

Requisito Funcional 5

Servicio de catering (RF_005)

- Recepción de información: El software debe recibir la solicitud de servicio de catering por parte de un cliente empresarial.
- Generación de salidas: Activación del servicio de catering para el restaurante seleccionado si está disponible.

Requisito Funcional 6

Gestionar usuarios (RF_006)

- Recepción de información: El software debe recibir solicitudes de consulta de usuarios.
- Generación de salidas: Presentación de la lista de usuarios registrados en la interfaz correspondiente

Requisito Funcional 7

Búsqueda de restaurantes (RF_007)

- Recepción de información: El software debe recibir criterios de búsqueda de restaurantes por parte de los clientes.
- Generación de salidas: Presentación de resultados de búsqueda según los criterios especificados.

Requisito Funcional 8

Filtrar restaurantes (RF_008)

- Recepción de información: El software debe recibir criterios de filtrado de restaurantes por parte de los clientes.
- Generación de salidas: Presentación de resultados de filtrado según los criterios especificados.

Requisito Funcional 9

Seleccionar comidas (RF_009)

- Recepción de información: El software debe recibir selecciones de comidas por parte de los clientes.
- Generación de salidas: Actualización del carrito de compras con las selecciones realizadas.

Requisito Funcional 11 Realizar pedidos

Seleccionar comidas (RF_011)

- Los clientes deben poder efectuar el pago para cerrar el pedido que deseen hacer. Después de completar el pago, deben recibir una confirmación del pedido realizado.

Entrada: Proceso de pago
Salida: Confirmación de pedido realizado

Requisito Funcional 20 Guardar datos

Guardar datos (RF_020)

- Después de cada uso de la aplicación, todos los datos y cambios realizados deben guardarse automáticamente. Al volver a ingresar en la aplicación, los datos del último uso deben estar disponibles.
  Estos requisitos describen las acciones básicas que el sistema debe realizar en respuesta a las interacciones del usuario.

Entrada: Datos y cambios efectuados dentro de la aplicación
Salida: Datos guardados y disponibles para próximo uso

Requisito Funcional 28

Opinion de restaurantes (RF_028)

- El usuario podrá añadir una opinión y una valoración de cada restaurante lo que podrá afectar en la calificación del mismo.

Entrada: El usuario pone su opinión y su valoración
Salida: El sistema añade la opinión y hace una media de las calificaciones añadiendo la nueva

Requisitos no funcionales

Datos

Interfaz Gráfica (RNF_015)

- Descripción: La aplicación se realizará con una interfaz desarrollada con Swing.
- Especificación de datos y formato: La interfaz gráfica implica la interacción con datos relacionados con la interfaz de usuario.

Seguridad de acceso (RNF_017)

- Descripción: Si las credenciales introducidas en el inicio de sesión no coinciden con las de algún usuario ya registrado, no dejará acceder a la aplicación.
- Especificación de datos y formato: Implica la manipulación de credenciales de inicio de sesión, como nombres de usuario y contraseñas.

Guardar datos (RNF_020)

- Descripción: Al acabar de usarse la aplicación se deben guardar todos los datos y cambios efectuados, y al volver a entrar en la aplicación todos los datos del último uso estarán disponibles.
- Especificación de datos y formato: Este requisito implica la manipulación y almacenamiento de diversos tipos de datos, que pueden incluir información sobre clientes, restaurantes, comidas, pedidos, entre otros. El formato de almacenamiento puede ser en archivos o en una base de datos.

Serialización (RNF_024)

- Descripción: Se deberán almacenar los datos en archivos, no se permiten el uso de bases de datos.
- Especificación de datos y formato: Implica la manipulación de datos para almacenarlos en archivos en lugar de bases de datos. El formato de estos archivos dependerá de los tipos de datos que se almacenen y puede variar según las necesidades del sistema.

Rendimiento

Calcular precios (RNF_010)

- Descripción: Una vez se añadan comidas al carrito, el sistema debe calcular el precio total del pedido antes de realizarlo. Aunque no se especifican métricas de rendimiento específicas, este requisito implica que el cálculo debe realizarse de manera eficiente para garantizar una experiencia de usuario fluida.

Tamaño mínimo para las pruebas (RNF_026)

- Descripción: Para verificar el correcto funcionamiento del sistema, se debe poder iniciar con un mínimo de 3 usuarios, 4 restaurantes y 12 comidas (3 por restaurante).

Seguridad

Seguridad de acceso (RNF_017)

- Descripción: Este requisito especifica que el sistema debe asegurar que solo los usuarios con credenciales válidas tengan acceso al sistema. Si las credenciales proporcionadas no coinciden con las de ningún usuario registrado, el acceso debe ser denegado.

Fiabilidad

Calcular precios (RNF_010)

- Una vez se añadan comidas al carrito se calculará el precio total del pedido antes de realizarlo. Este requisito asegura que el cálculo de precios se realice correctamente y de manera consistente para cada pedido.

Generar facturas (RNF_013)

- Al realizar el pedido se imprimirá una factura del mismo. Este requisito garantiza que se generen facturas de forma fiable y precisa para cada pedido realizado en el sistema.

Almacenar pedidos (RNF_014)

- Se guardarán todos los datos de cada venta. Este requisito asegura que los datos de los pedidos se almacenen de manera fiable para su posterior referencia o análisis.

Excepciones (RNF_021)

- Se deberá realizar un correcto tratamiento de excepciones. Este requisito asegura que el sistema maneje adecuadamente cualquier excepción que pueda surgir durante su ejecución, evitando fallos inesperados y garantizando una experiencia de usuario más estable.

Disponibilidad

Calcular precios (RNF_010)

- El sistema debe estar disponible el 100% del tiempo para calcular los precios de los pedidos. Esto garantiza que los usuarios siempre puedan ver con precisión el precio total antes de confirmar sus pedidos.

Realizar pedido (RF_011)

- El sistema debe estar disponible el 100% del tiempo durante el proceso de realizar un pedido, ya que los clientes deben poder efectuar el pago en cualquier momento sin experimentar tiempo de inactividad.

Generar facturas (RNF_013)

- El sistema debe estar disponible el 100% del tiempo para generar facturas. Esto es crucial para que los usuarios reciban su factura inmediatamente después de realizar un pedido, sin experimentar tiempo de inactividad en la generación de la factura.

Almacenar pedidos (RNF_014)

- El sistema debe estar disponible el 100% del tiempo para almacenar los datos de cada venta. Es fundamental que cada venta se registre de manera oportuna y precisa, sin experimentar tiempo de inactividad en el proceso de almacenamiento de datos.

Mantenibilidad

Interfaz gráfica (RNF_015)

- Los desarrolladores deben realizar tareas de mantenimiento para mantener la interfaz gráfica actualizada y funcional.

- Se espera que las actualizaciones y mejoras se realicen según sea necesario para garantizar que la interfaz siga siendo intuitiva y fácil de usar.

Fácil mantenibilidad (RNF_025)

- Cualquier programador debe poder mantener el sistema fácilmente en Java, empleando comentarios y una buena organización para facilitar el entendimiento.

- El mantenimiento debe ser continuo y que se deben realizar actualizaciones y mejoras según sea necesario para garantizar la facilidad de mantenimiento a lo largo del tiempo.

Otros requisitos

Descurnto para empresas (RNF_012)

- Este requisito establece que las empresas obtendrán un 10% de descuento en las transacciones realizadas.
- Es una especificación de negocio que afecta a la política de precios y transacciones.

Lenguaje de Java (RNF_018)

- Este requisito establece que la aplicación debe estar desarrollada en el lenguaje Java y ser compatible.
- Afecta al desarrollo y al entorno de ejecución del sistema.

Separación de clases (RNF_023)

- Las clases del sistema deben ser separadas por paquetes según su función, como clases de dominio, gestión de interfaz y modelo.
- Es una especificación de diseño y estructura que contribuye a la mantenibilidad del sistema a largo plazo.

Tamaño mínimo para las pruebas (RNF_026)

- Este requisito establece el tamaño mínimo necesario para realizar pruebas de funcionamiento del sistema, especificando la cantidad mínima de usuarios, restaurantes y comidas que deben estar disponibles.
- Es una especificación que aborda aspectos de calidad y prueba, pero no se centra directamente en las funcionalidades del sistema.

UML (RNF_027)

- Este requisito implica la necesidad de realizar un diagrama UML mediante ingeniería inversa. Esto sugiere que se debe documentar la estructura y relaciones del sistema utilizando UML.

3 DOCUMENTOS DE CASOS DE USO

CASO DE USO: CU_001 – Registro de Cliente Particular

Objetivo:
Permitir que un cliente particular se registre en la aplicación proporcionando sus datos personales.

Inicio:
El actor (Cliente Particular) selecciona la opción de registro en la interfaz de usuario.

Flujo Principal:

Actor	Sistema
1.- El actor selecciona la opción de registro.	2.- El sistema muestra un formulario de registro con campos para DNI, nombre, correo electrónico, clave, dirección, tarjeta de crédito y teléfono.
3.- El actor ingresa los datos solicitados.	4.- El sistema valida los datos y registra al cliente particular en la aplicación.

Flujos Excepcionales:
En el paso 3: Si algún dato ingresado no cumple con el formato requerido, el sistema muestra un mensaje de error y solicita al actor corregirlo.

Precondiciones:
El actor no debe estar registrado previamente en la aplicación.

Postcondiciones:
El cliente particular queda registrado en la aplicación con sus datos personales.

Finalización:
El sistema muestra un mensaje de confirmación de registro y redirige al actor a la pantalla de inicio de sesión.

Diagrama Complementarios:
Referencia al diagrama de actividad de registro.

Requisitos funcionales:
RF_001 - Registro de Cliente Particular.

Requisitos especiales:
Etiquetas a los requisitos <u>no funcionales</u> (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:

CASO DE USO: CU_002 – Login de Cliente

Objetivo:
Permitir que un cliente (particular o empresa) inicie sesión en la aplicación.

Inicio:
El actor (Cliente) selecciona la opción de iniciar sesión en la interfaz de usuario.

Flujo Principal:

Actor	Sistema
1.- El actor selecciona la opción de iniciar sesión.	2.- El sistema muestra un formulario con campos para correo electrónico y clave.
3.- El actor ingresa sus credenciales.	4.- El sistema valida las credenciales y permite el acceso al sistema.

Flujos Excepcionales:

En el paso 3: Si las credenciales ingresadas son incorrectas, el sistema muestra un mensaje de error.

Precondiciones:

El actor debe estar registrado en la aplicación.

Postcondiciones:

El cliente queda autenticado en el sistema.

Finalización:

El sistema redirige al cliente a la pantalla principal de la aplicación.

Diagrama Complementarios:

Referencia al diagrama de actividad de inicio de sesión.

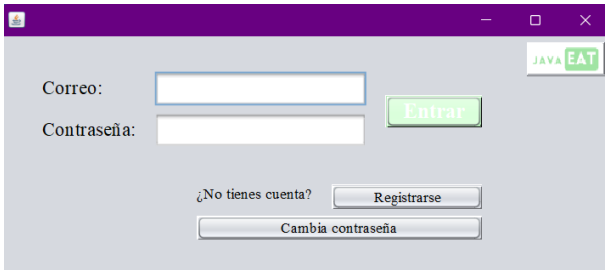
Requisitos funcionales:

RF_002 - Login de Cliente.

Requisitos especiales:

Etiquetas a los requisitos <u>no funcionales</u> (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:



CASO DE USO: CU_003 – Búsqueda de Restaurantes por Código Postal y por especialidad

Objetivo:
Permitir que un cliente busque restaurantes cercanos según su código postal.

Inicio:
El actor (Cliente) selecciona la opción de búsqueda de restaurantes en la interfaz de usuario.

Flujo Principal:

Actor	Sistema
1.- El actor selecciona la opción de búsqueda de restaurantes.	2.- El sistema solicita al actor ingresar su código postal.
3.- El actor ingresa su código postal.	4.- El sistema filtra y muestra la lista de restaurantes cercanos basándose en los 4 primeros dígitos del código postal.
5.- El actor pulsa en la caja de especialidades	6.- El sistema muestra las especialidades que ofrece
7.- El actor selecciona una especialidad	8.- El sistema filtra y muestra la lista de restaurantes que se dedican a esa especialidad en función de la lista anterior con el código postal

Flujos Excepcionales:

En el paso 3: Si el código postal ingresado no cumple con el formato requerido, el sistema muestra un mensaje de error y solicita al actor corregirlo.

Precondiciones:
El actor debe estar autenticado en el sistema.

Postcondiciones:
Se muestra la lista de restaurantes cercanos según el código postal ingresado.

Finalización:

El sistema presenta al actor la lista de restaurantes y opciones de filtrado.

Diagrama Complementarios:

Referencia al diagrama de actividad de búsqueda de restaurantes.

Requisitos funcionales:

RF_007 - Búsqueda de Restaurantes
RF_008 – Filtrar restaurantes

Requisitos especiales:

Etiquetas a los requisitos no funcionales (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:

CASO DE USO: CU_004 – Selección de Restaurante y Comidas

Objetivo:
Permitir que un cliente seleccione un restaurante y las comidas que desea comprar.

Inicio:
El actor (Cliente) elige un restaurante de la lista de restaurantes cercanos.

Flujo Principal:

Actor	Sistema
1.- El actor selecciona un restaurante de la lista.	2.- El sistema muestra el menú del restaurante con las comidas disponibles.
3.- El actor elige las comidas y la cantidad que desea comprar.	4.- El sistema agrega las comidas al "carrito de la compra".

Flujos Excepcionales:
En el paso 3: Si el actor intenta seleccionar una cantidad negativa, el sistema muestra un mensaje de error.

Precondiciones:

El actor debe haber realizado la búsqueda de restaurantes y seleccionado uno.

Postcondiciones:

Las comidas seleccionadas quedan registradas en el "carrito de la compra".

Finalización:

El sistema muestra al actor el contenido actual del "carrito de la compra".

Diagrama Complementarios:

Referencia al diagrama de actividad de selección de restaurante y comidas.

Requisitos funcionales:

RF_009 - Selección de Comidas.

Requisitos especiales:

Etiquetas a los requisitos no funcionales (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:

CASO DE USO: CU_005 – Proceso de Pago

Objetivo:
Permitir que un cliente realice el pago de las comidas seleccionadas en el "carrito de la compra".

Inicio:
El actor (Cliente) decide proceder al pago desde la pantalla del "carrito de la compra".

Flujo Principal:

Actor	Sistema
1.- El actor selecciona la opción de proceder al pago desde el "carrito de la compra".	2.- El sistema muestra el resumen de la compra, incluyendo el total a pagar.
3.- El actor elige el método de pago y proporciona la información correspondiente.	4.- El sistema valida la información de pago y realiza la transacción.

Flujos Excepcionales:
En el paso 3: Si la información de pago no es válida, el sistema muestra un mensaje de error y solicita al actor corregirla.

Precondiciones:

El actor debe haber seleccionado al menos una comida en el "carrito de la compra".

Postcondiciones:

Se realiza la transacción y se generan los registros de la venta.

Finalización:

El sistema muestra un mensaje de confirmación de la compra y redirige al actor a la pantalla principal.

Diagrama Complementarios:

Referencia al diagrama de actividad de proceso de pago.

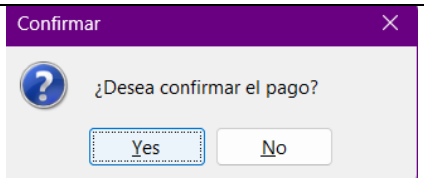
Requisitos funcionales:

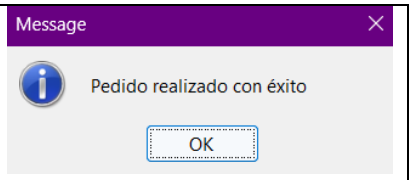
RF_011 – Realizar pedido.

Requisitos especiales:

Etiquetas a los requisitos <u>no funcionales</u> (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:





CASO DE USO: CU_006 – Opinión sobre Restaurantes

Objetivo:

Permitir que un cliente opine sobre un restaurante después de realizar una compra.

Inicio:

El actor (Cliente) selecciona la opción de opinar sobre un restaurante desde la pantalla de historial de compras.

Flujo Principal:

Actor	Sistema
1.- El actor selecciona la opción de opinar sobre un restaurante.	2.- El sistema muestra la lista de restaurantes previamente comprados por el cliente.
3.- El actor elige el restaurante sobre el cual desea opinar.	4.- El sistema muestra un formulario para que el actor ingrese su opinión y calificación del restaurante.
5.- El actor proporciona su opinión y calificación.	6.- El sistema registra la opinión y actualiza la calificación del restaurante.

Flujos Excepcionales:
En el paso 5: Si el actor intenta dejar una opinión sin calificación, el sistema muestra un mensaje de error.

Precondiciones:

El actor debe haber realizado al menos una compra.

Postcondiciones:

La opinión y calificación quedan registradas en el sistema.

Finalización:

El sistema muestra un mensaje de confirmación de la opinión.

Diagrama Complementarios:

Referencia al diagrama de actividad de opinión sobre restaurantes.

Requisitos funcionales:

RF_028: Opinion de restaurantes

Requisitos especiales:

Etiquetas a los requisitos <u>no funcionales</u> (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:

Imágenes de las pantallas de la aplicación que aparecen a lo largo del desarrollo del caso.

CASO DE USO: CU_007 – Modificación de Datos Personales

Objetivo:
Permitir que un cliente modifique sus datos personales registrados en la aplicación.

Inicio:
El actor (Cliente) selecciona la opción de modificar datos personales desde la pantalla de perfil.

Flujo Principal:

Actor	Sistema
1.- El actor selecciona la opción de modificar datos personales.	2.- El sistema muestra un formulario con los datos personales actuales del cliente.
3.- El actor realiza las modificaciones necesarias en los campos deseados.	4.- El sistema valida los cambios y actualiza la información del cliente.

Flujos Excepcionales:
En el paso 3: Si algún dato modificado no cumple con el formato requerido, el sistema muestra un mensaje de error y solicita al actor corregirlo.

Precondiciones:

El actor debe estar autenticado en el sistema.

Postcondiciones:

Los datos personales del cliente quedan actualizados en el sistema.

Finalización:

El sistema muestra un mensaje de confirmación de la modificación y redirige al actor a la pantalla de perfil actualizada.

Diagrama Complementarios:

Referencia al diagrama de actividad de modificación de datos personales.

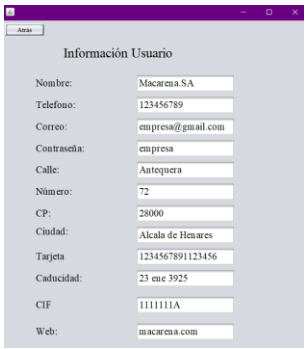
Requisitos funcionales:

RF_006 : Gestion de usuarios

Requisitos especiales:

Etiquetas a los requisitos no funcionales (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:



CASO DE USO: CU_008 - Búsqueda de Restaurantes con Servicio de Catering

Objetivo:
Permitir que una empresa cliente busque restaurantes que ofrezcan servicio de catering.

Inicio:
El actor (Empresa Cliente) selecciona la opción de búsqueda de restaurantes con servicio de catering.

Flujo Principal:

Actor	Sistema
1.- El actor selecciona la opción de búsqueda de restaurantes con servicio de catering.	2.- El sistema muestra la lista de restaurantes que ofrecen servicio de catering.
3.- El actor elige un restaurante de la lista.	4.- El sistema muestra los servicios de ese restaurate.

Flujos Excepcionales:
En el paso 2: Si no hay restaurantes con servicio de catering, el sistema muestra un mensaje indicando que no hay resultados.

Precondiciones:

El actor debe ser una empresa cliente.

Postcondiciones:

Se muestra la lista de restaurantes con servicio de catering.

Finalización:

El sistema presenta al actor la lista de restaurantes y opciones de filtrado.

Diagrama Complementarios:

Referencia al diagrama de actividad, secuencias o comunicación que complementa la información del caso de uso y que se incluye en otra parte del documento.

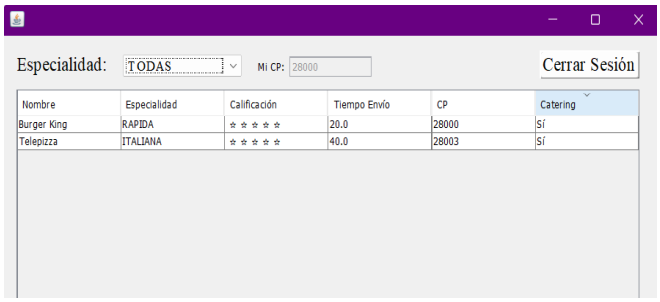
Requisitos funcionales:

RF_005: Servicio de Catering

Requisitos especiales:

Etiquetas a los requisitos <u>no funcionales</u> (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:



CASO DE USO: CU_009 – Selección de Catering

Objetivo:
Permitir que una empresa cliente seleccione el servicio de catering deseado.

Inicio:
El actor (Empresa Cliente) elige un restaurante de la lista de restaurantes con servicio de catering.

Flujo Principal:

Actor	Sistema
1.- El actor selecciona un restaurante de la lista.	2.- El sistema muestra las opciones de catering disponibles para ese restaurante.
3.- El actor elige las comidas y servicios deseados para el catering.	4.- El sistema muestra un resumen del catering con el precio total.

Flujos Excepcionales:
En el paso 3: Si el actor intenta seleccionar una cantidad negativa o no elige ningún servicio, el sistema muestra un mensaje de error.

Precondiciones:

El actor debe haber realizado la búsqueda de restaurantes con servicio de catering y seleccionado uno.

Postcondiciones:

El catering queda registrado en la aplicación.

Finalización:

El sistema muestra al actor el resumen del catering y opciones de proceder al pago.

Diagrama Complementarios:

Referencia al diagrama de actividad de selección de catering.

Requisitos funcionales:

RF_005: Servicio de Catering

Requisitos especiales:

Etiquetas a los requisitos no funcionales (seguridad, rendimiento, etc.) que afectan específicamente al caso.

Descripción de Interfaz:

Imágenes de las pantallas de la aplicación que aparecen a lo largo del desarrollo del caso.

CASO DE USO: CU_010 – Consulta de Ventas Realizadas por el Administrador

Objetivo:

Permitir que el administrador de la aplicación consulte las ventas realizadas.

Inicio:

El actor (Administrador) selecciona la opción de consulta de ventas desde el panel de administrador.
Flujo Principal:

Actor	Sistema
1.- El actor selecciona la opción de consulta de ventas.	2.- El sistema muestra la lista de todas las ventas realizadas.
3.- El actor puede seleccionar todas las ventas o filtrar por una fecha específica.	

Flujos Excepcionales:
En el paso 2: Si no hay ventas registradas, el sistema muestra un mensaje indicando que no hay resultados.

Precondiciones:

El actor debe estar autenticado como administrador.

Postcondiciones:

Se muestra la lista de ventas realizadas.

Finalización:

El sistema presenta al administrador la lista de ventas y opciones de filtrado.

Diagrama Complementarios:

Referencia al diagrama de actividad, secuencias o comunicación que complementa la información del caso de uso y que se incluye en otra parte del documento.

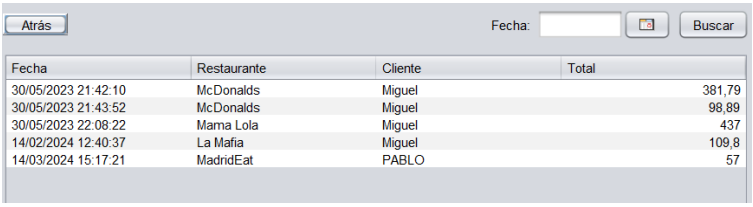
Requisitos funcionales:

RF_ 0013: Generar Facturas
RF_014: Almacenar Pedidos
RF_20: Guardar datos

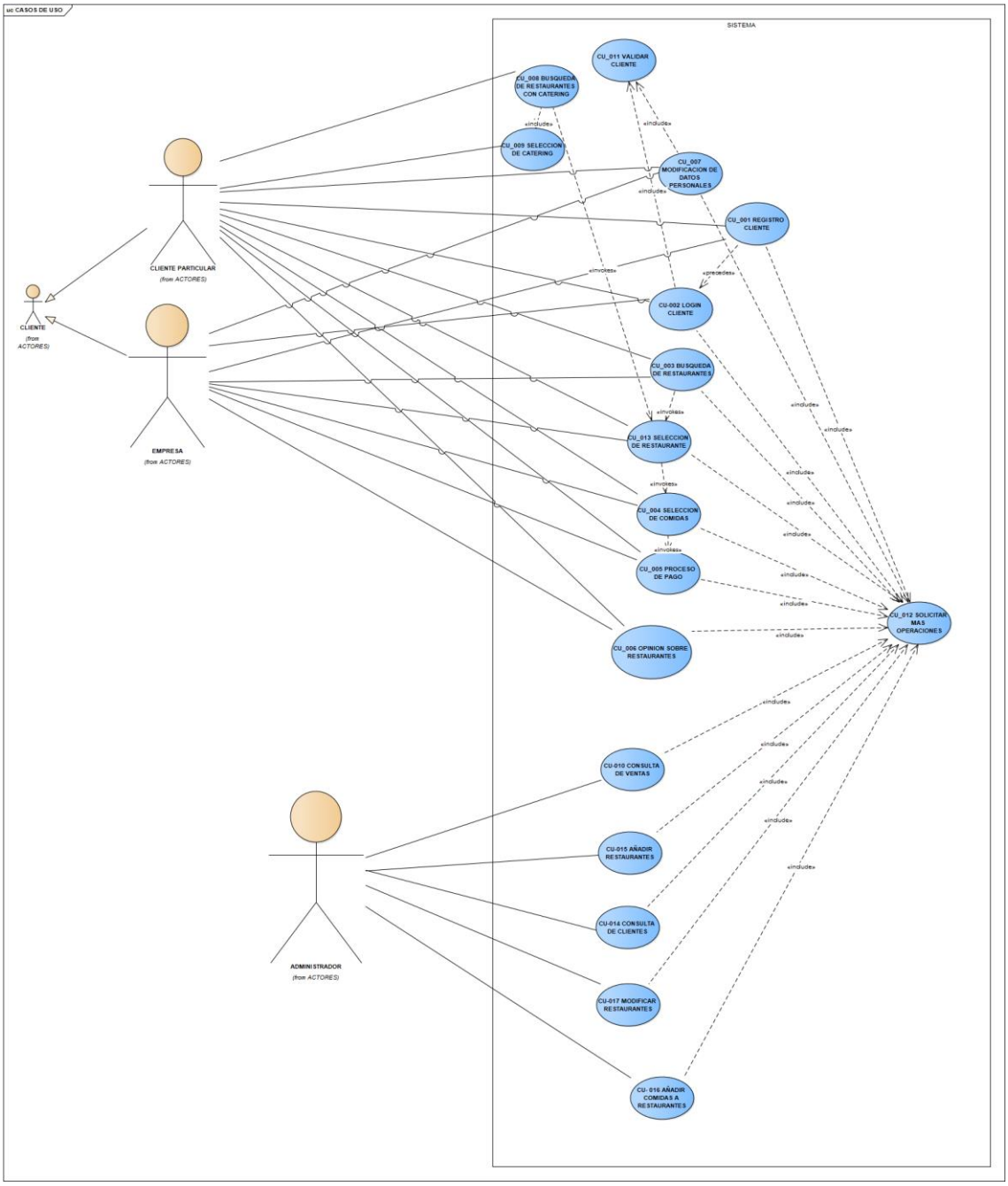
Requisitos especiales:

Etiquetas a los requisitos <u>no funcionales</u> (seguridad, rendimiento, etc.) que afectan específicamente al caso.

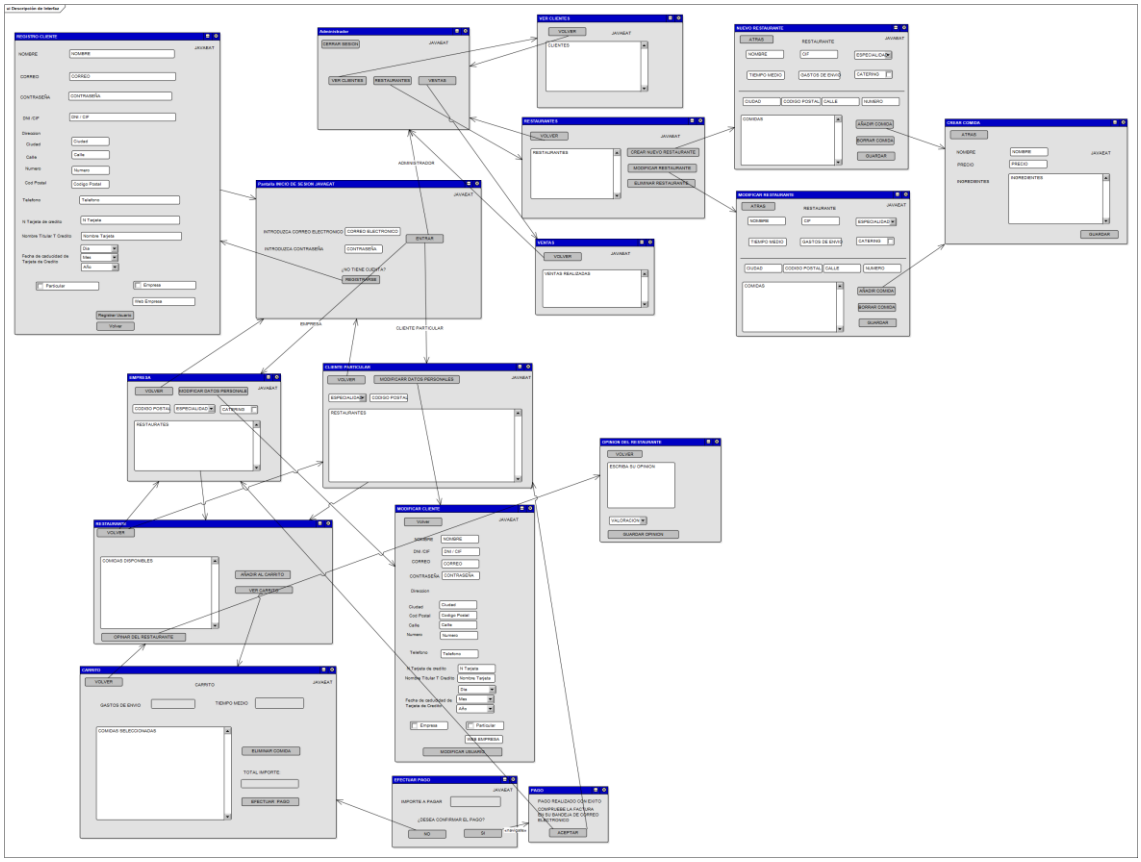
Descripción de Interfaz:



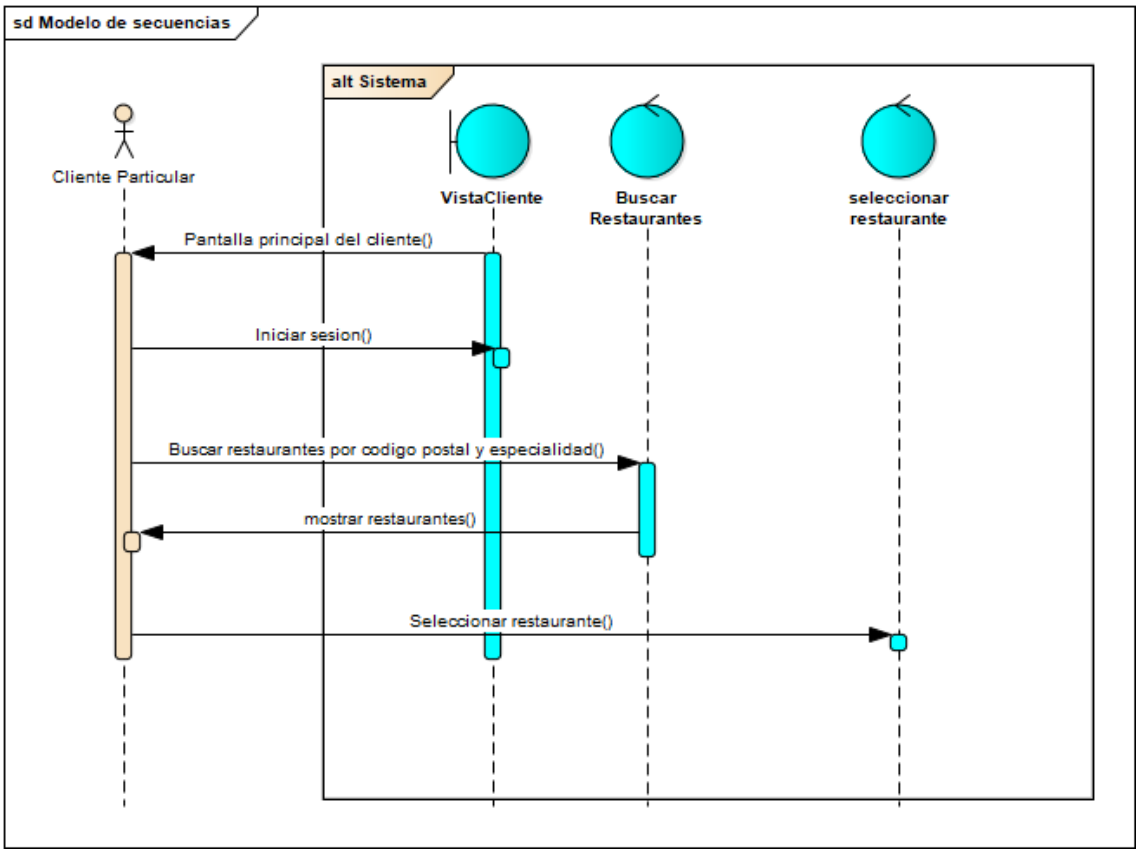
4 DIAGRAMAS DE CASO DE USO



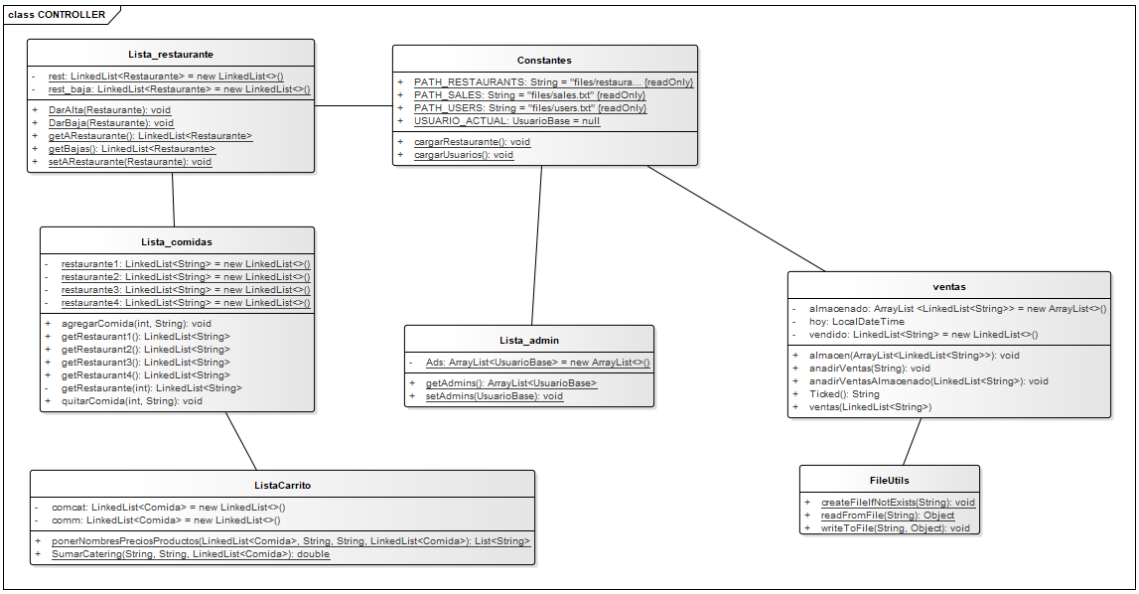
5 DIAGRAMA DE INTERFAZ

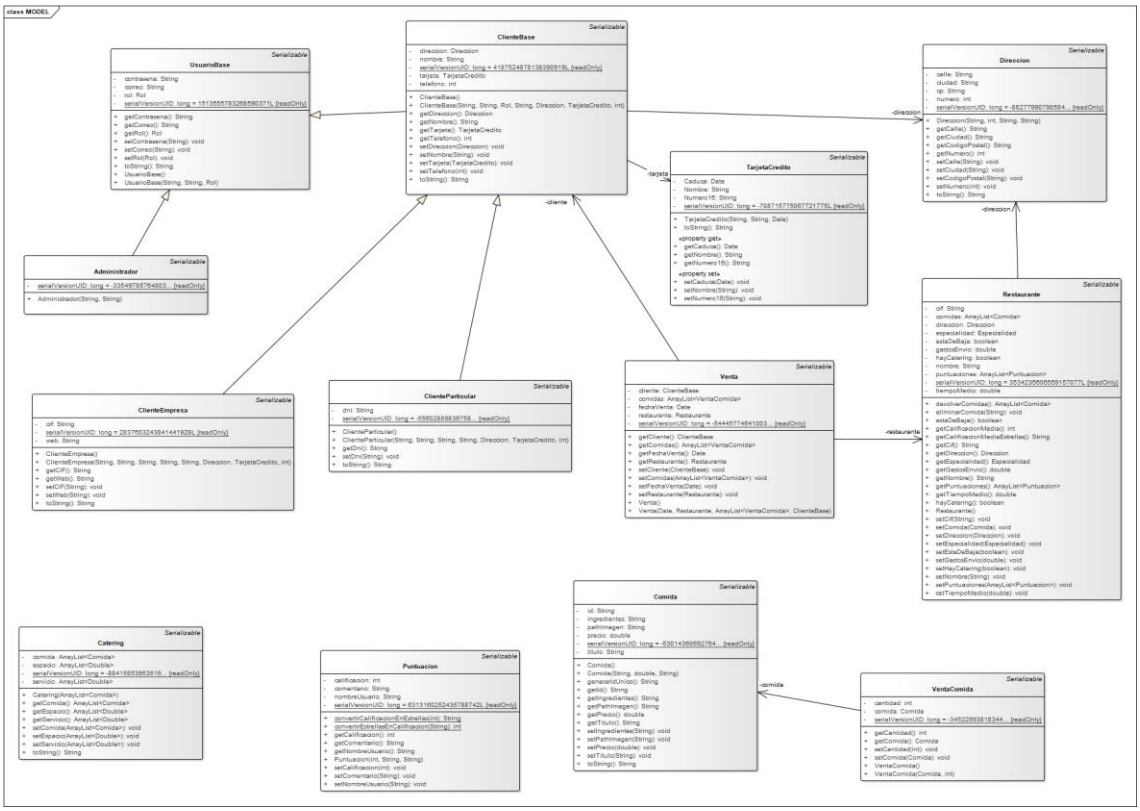


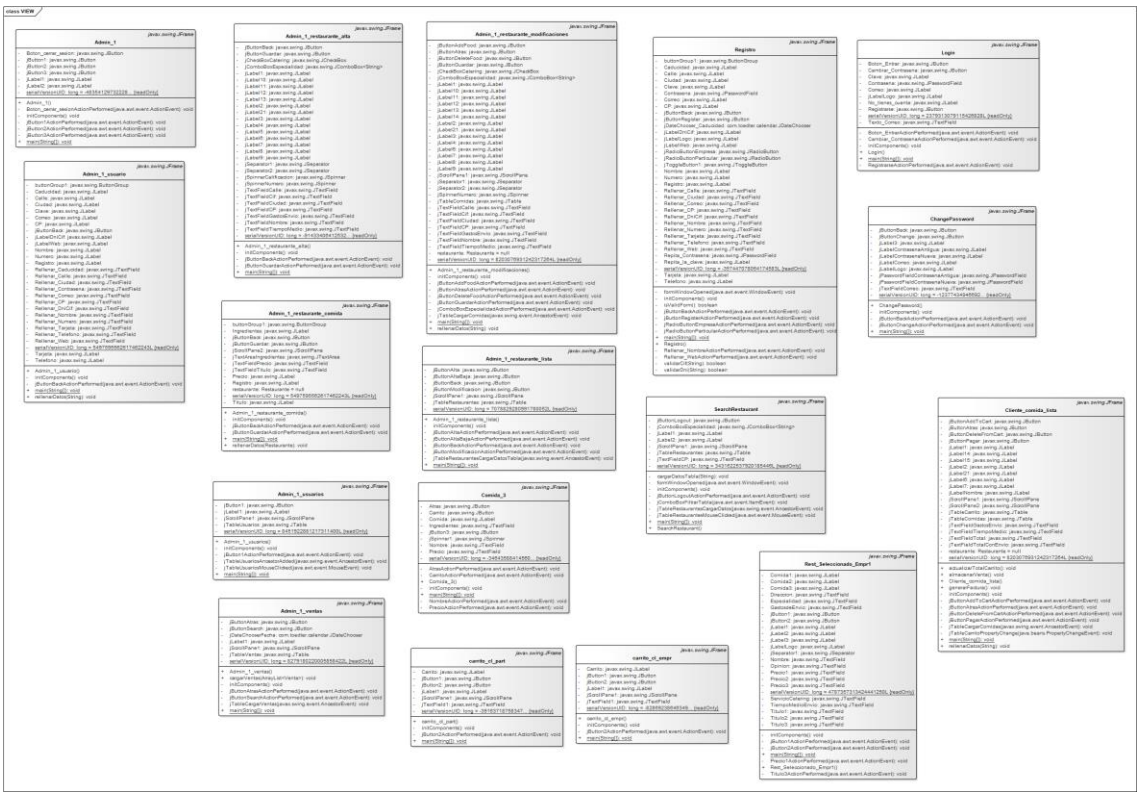
6 DIAGRAMA DE SECUENCIAS



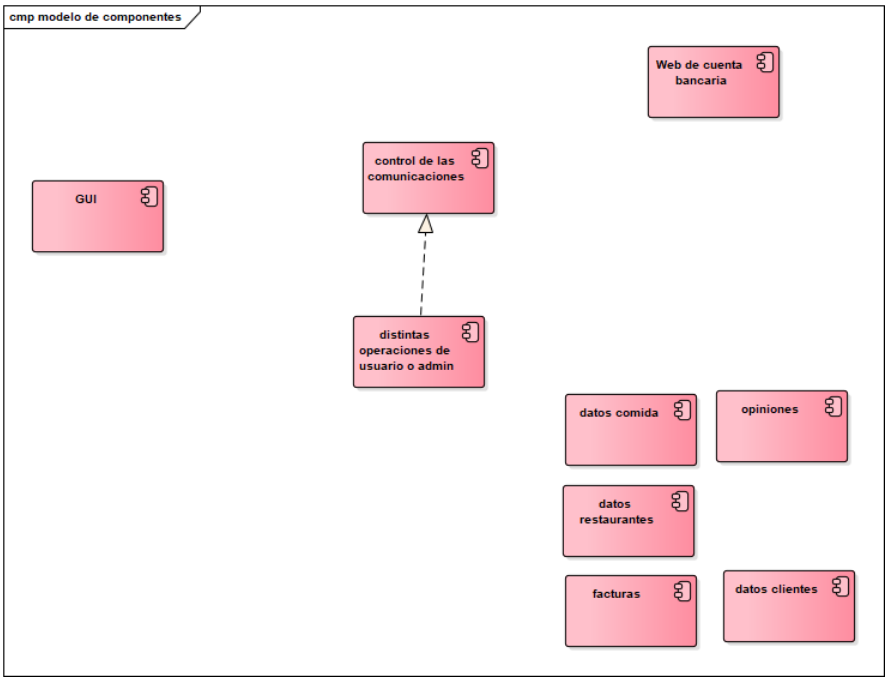
7 DIAGRAMA DE CLASES DE DISEÑO

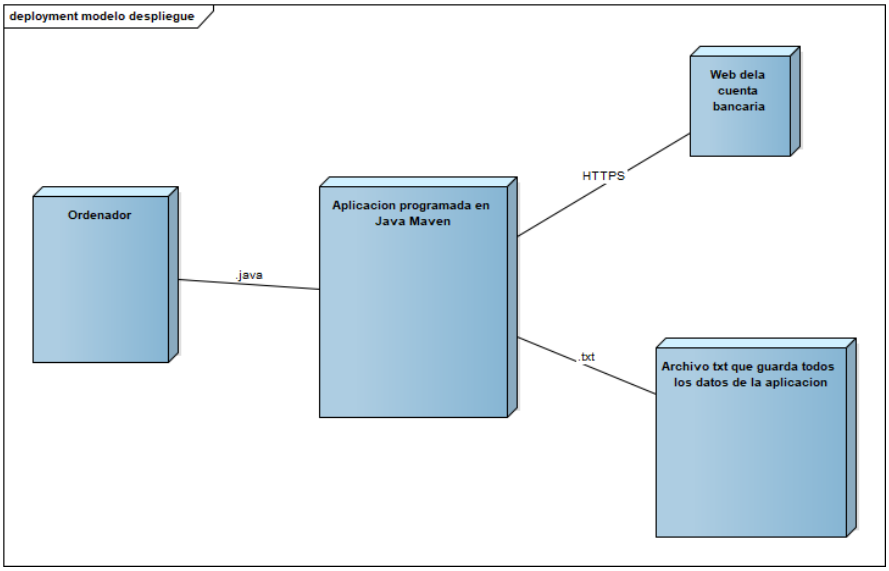


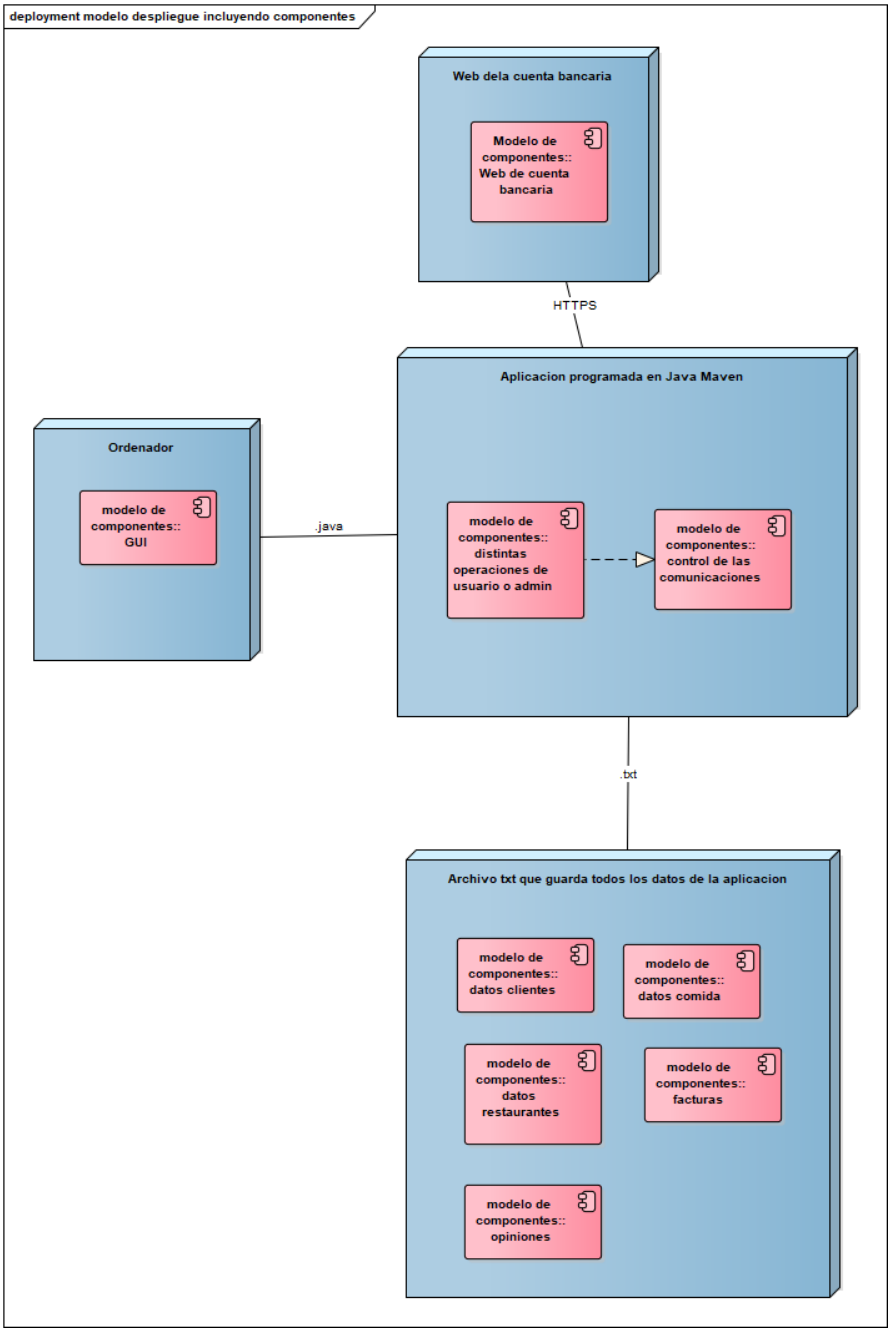

class MODEL
UsuarioBase
contraseña: String
correo: String
rol: Rol
serialVersionUID: long = 1513555783268590371L [readOnly]
getContraseña(): String
getCorreo(): String
getRol(): Rol
setContraseña(String): void
setCorreo(String): void
setRol(Rol): void
toString(): String
UsuarioBase()
UsuarioBase(String, String, Rol)
ClienteBase
direccion: Direccion
nombre: String
serialVersionUID: long = 4187524878138980919L [readOnly]
tarjeta: TarjetaCredito
telefono: int
ClienteBase()
ClienteBase(String, String, Rol, String, Direccion, TarjetaCredito, int)
getDireccion(): Direccion
getNombre(): String
getTarjeta(): TarjetaCredito
getTelefono(): int
setDireccion(Direccion): void
setNombre(String): void
setTarjeta(TarjetaCredito): void
setTelefono(int): void
toString(): String
Direccion
calle: String
ciudad: String
cp: String
numero: int
serialVersionUID: long = -8527799079584... [readOnly]
Direccion(String, int, String, String)
getCalle(): String
getCiudad(): String
getCodigoPostal(): String
getNumero(): int
setCalle(String): void
setCiudad(String): void
setCodigoPostal(String): void
setNumero(int): void
toString(): String
-direccion
-direccion
TarjetaCredito
Caducidad: Date
Nombre: String
Numero16: String
serialVersionUID: long = -708715759087721775L [readOnly]
TarjetaCredito(String, String, Date)
toString(): String
«property get»
getCaducidad(): Date
getNombre(): String
getNumero16(): String
«property set»
setCaducidad(Date): void
setNombre(String): void
setNumero16(String): void
-tarjeta
Administrador
serialVersionUID: long = -3354978578483... [readOnly]
Administrador(String, String)
Restaurante
cif: String
comidas: ArrayList<Comida>
direccion: Direccion
especialidad: Especialidad
estaDeBaja: boolean
gastosEnvio: double
hayCatering: boolean
nombre: String
puntuaciones: ArrayList<Puntuacion>
serialVersionUID: long = 3534235505559157075L [readOnly]
tiempoMedio: double
devolverComida(): ArrayList<Comida>
eliminarComida(String): void
estaDeBaja(): boolean
getCalificacionMedia(): int
getCalificacionMediaEstrellas(): String
getCif(): String
getDireccion(): Direccion
getEspecialidad(): Especialidad
getGastosEnvio(): double
getNombre(): String
getPuntuaciones(): ArrayList<Puntuacion>
getTiempoMedio(): double
hayCatering(): boolean
Restaurante()
setCif(String): void
setComidas(Comida): void
setDireccion(Direccion): void
setEspecialidad(Especialidad): void
setEstaDeBaja(boolean): void
setGastosEnvio(double): void
setHayCatering(boolean): void
setNombre(String): void
setPuntuaciones(ArrayList<Puntuacion>): void
setTiempoMedio(double): void
-restaurante
ClienteEmpresa
cif: String
serialVersionUID: long = 2837503243841441928L [readOnly]
web: String
ClienteEmpresa()
ClienteEmpresa(String, String, String, String, String, Direccion, TarjetaCredito, int)
getCIF(): String
getWeb(): String
setCIF(String): void
setWeb(String): void
toString(): String
ClienteParticular
dni: String
serialVersionUID: long = -5660289983678... [readOnly]
ClienteParticular()
ClienteParticular(String, String, String, String, Direccion, TarjetaCredito, int)
getDni(): String
setDni(String): void
toString(): String
-cliente
Venta
cliente: ClienteBase
comidas: ArrayList<VentaComida>
fechaVenta: Date
restaurante: Restaurante
serialVersionUID: long = -5444577464103... [readOnly]
getCliente(): ClienteBase
getComidas(): ArrayList<VentaComida>
getFechaVenta(): Date
getRestaurante(): Restaurante
setCliente(ClienteBase): void
setComidas(ArrayList<VentaComida>): void
setFechaVenta(Date): void
setRestaurante(Restaurante): void
Venta()
Venta(Date, Restaurante, ArrayList<VentaComida>, ClienteBase)
Comida
id: String
ingredientes: String
pathImagen: String
precio: double
serialVersionUID: long = -6301436869278... [readOnly]
titulo: String
Comida()
Comida(String, double, String)
getpathUnico(): String
getId(): String
getingredientes(): String
getPathImagen(): String
getPrecio(): double
getTitulo(): String
setingredientes(String): void
setPathImagen(String): void
setPrecio(double): void
setTitulo(String): void
toString(): String
-comida
VentaComida
cantidad: int
comida: Comida
serialVersionUID: long = -3452299381844... [readOnly]
getCantidad(): int
getComida(): Comida
setCantidad(int): void
setComida(Comida): void
VentaComida()
VentaComida(Comida, int)
Catering
comida: ArrayList<Comida>
espacio: ArrayList<Double>
serialVersionUID: long = -8841565393616... [readOnly]
servicio: ArrayList<Double>
Catering(ArrayList<Comida>)
getComida(): ArrayList<Comida>
getEspacio(): ArrayList<Double>
getServicio(): ArrayList<Double>
setComida(ArrayList<Comida>): void
setEspacio(ArrayList<Double>): void
setServicio(ArrayList<Double>): void
toString(): String
Puntuacion
calificacion: int
comentario: String
nombreUsuario: String
serialVersionUID: long = 6313160252435788742L [readOnly]
convertirCalificacionEnEstrellas(int): String
convertirEstrellasEnCalificacion(String): int
getCalificacion(): int
getComentario(): String
getNombreUsuario(): String
Puntuacion(int, String, String)
setCalificacion(int): void
setComentario(String): void
setNombreUsuario(String): void


class VIEW
Admin_1
Admin_1_usuario
Admin_1_restaurante_alta
Admin_1_restaurante_modificaciones
Registro
Login
ChangePassword
Admin_1_restaurante_comida
Admin_1_restaurante_baja
SearchRestaurant
Cliente_comida_lista
Admin_1_usuarios
Comida_3
Admin_1_ventas
carrito_cl_part
carrito_cl_empr
Rest_Seleccionado_Empr1

8 DIAGRAMAS DE COMPONENTES Y DESPLIEGUE







9 TRAZABILIDAD DE UN REQUISITO FUNCIONAL EN EL PROYECTO

TRAZABILIDAD DE Registro de cliente (RF_001)

El usuario que quiere iniciar sesión no lo podrá hacer sin registrarse. En el momento en el que pulse el botón de registrarse aparecerá una pantalla con todos los datos que debe meter y si es una empresa o un particular. Una vez introduzca los datos se comprobara que no hay ningún usuario con los mismos datos y si es asi se registraran las credenciales y los datos en el txt de la aplicación.

10 CONCLUSIONES

Durante la realización de este trabajo, hemos aprendido como se hace un software desde cero para que el cliente final obtenga exactamente lo que quiere:

La especificación de requisitos nos ha permitido comprender a fondo las necesidades del usuario y definir claramente las funcionalidades que nuestro sistema debe cumplir. Esta etapa fue fundamental para establecer una base sólida para el desarrollo posterior.

La documentación de casos de uso nos llevó a profundizar en los diferentes escenarios de interacción entre el usuario y el sistema. Al identificar y describir estos casos, pudimos visualizar con mayor claridad cómo se utilizará el software en la práctica.

El diseño de la interfaz gráfica fue un aspecto crucial para garantizar una experiencia de usuario intuitiva y eficiente. Al crear un diagrama que muestra las pantallas propuestas y sus relaciones, pudimos visualizar la estructura general del sistema y anticipar la navegación del usuario.

Los diagramas de secuencias y clases nos ayudaron a comprender cómo interactúan los diferentes componentes del sistema y cómo se comunican entre sí. Este análisis profundo fue esencial para diseñar una arquitectura sólida y coherente.

La identificación de los componentes y la planificación del despliegue nos brindaron una visión completa del entorno tecnológico en el que operará nuestro sistema. Esto nos permitió tomar decisiones informadas sobre la infraestructura necesaria para implementar y ejecutar el software de manera efectiva.